



Lab Report-03

Course: CISC5950

Title: Big Data Computing

Submitted To:

Dr. Ying Mao

Associate Professor

Department of Computer and Information Science

Fordham University

Submitted By:

Muhammad Zawad Mahmud (ID# A23176102)

Samiha Islam (ID#A23171266)

Dolat (ID#A23205664)

Date of Submission: 04/22/2026

Part 1: Urban Analytics with Spark (NYC Parking Data)

Task 1.1: Data Cleaning Pipeline Modernization

How the Code Works

We created one Spark job that takes the place of the two MapReduce jobs from Lab 2. The code reads the raw CSV from /parking/raw/ using spark.read.csv with header=True. This handles the multi-mapper header problem that required positional indexing in Lab 2. It also defines an explicit 43-column StructType schema, which avoids schema inference overhead.

The cleaning pipeline uses all five Lab 2 rules as chained DataFrame transformations. For example, the extract_year_from_date helper uses regexp_extract and substr to get the year from both MM/DD/YYYY and ISO formats. Records are filtered to keep only 2024-2025 dates with non-null plate_id, violation_code, and issue_date. The normalize_color function uses a chained when() expression with over 60 abbreviation mappings. The normalize_state function uses a similar chained when() for state codes. Finally, the extract_hour function uses regexp_extract to pull the hour digits and AM/PM indicator from violation_time values. The report counts (using DataFrame .count() and .filter()) and the final cleaned output both go through the same pipeline. This means that there is no need for two separate jobs with intermediate HDFS writes.

I/O

Component	Format
Input	Raw CSV from /parking/raw/ (43 columns)
Output	Cleaned CSV to /parking/spark_clean/ + quality report to console

Data Quality Report

Metric	Count	Percentage
Total input records	8,279,622	-
Valid output records	8,278,399	100.0%
Invalid date records	525	0.006%
Missing critical fields	698	0.008%
Color standardizations	6,180,973	74.7%
State code corrections	0	0.0%

```

=====
DATA QUALITY REPORT
=====
TOTAL_INPUT      8279622
VALID_OUTPUT_RECORDS  8278399 (100.0%)
INVALID_DATE_RECORDS  525      (0.006%)
MISSING_CRITICAL_FIELDS 698      (0.008%)
COLOR_STANDARDIZATIONS 6180973 (74.7%)
STATE_CODE_CORRECTIONS 0         (0.000%)
=====

```

Fig. 1: Task 1.1 Data Quality Report output.

Implementation Complexity Comparison

The Spark implementation is dramatically simpler than the Lab 2 MapReduce pipeline. Lab 2 required 2 separate MapReduce jobs (clean + quality) with 4 Python files (mapper_clean.py, reducer_clean.py, mapper_quality.py, reducer_quality.py) totaling roughly 250 lines of code, plus intermediate HDFS writes between jobs. The Spark version is a single 200-line Python file executing as one job with no intermediate writes. All transformations are chained as a single DAG that Spark's Catalyst optimizer plans and executes together. The multi-mapper header problem that required positional indexing in Lab 2 is eliminated entirely because Spark's CSV reader handles headers natively.

Task 1.2: Geographic Hotspot Analysis with Broadcast Joins

How the Code Works

This task gets the cleaned data from /parking/spark_clean/, which is the output of Task 1.1. We make a small fines lookup DataFrame in memory that links 28 violation codes to amounts of money (for example, code 14=\$115, code 21=\$45, and so on). The broadcast() function is the main Spark pattern. It wraps the small DataFrame and tells Spark to send it to the memory of every worker node instead of shuffling the large parking dataset. The join then happens on each worker's computer without any network shuffle. After the join, coalesce() provides a default \$50 fine for codes not in the lookup table. The Top-K aggregation is expressed as a single chain: groupBy(street_code, street_name).agg(count, sum, tickets_per_day).orderBy(desc).limit(20). Spark's Catalyst optimizer handles the partial aggregation and top-K selection automatically, replacing the manual local-top-K pattern from Lab 2. The result is saved to /parking/hotspots/.

I/O

Component	Format
Input	/parking/spark_clean/ + in-memory fines DataFrame

Output	/parking/hotspots/ (top 20 locations with tickets, revenue, tickets/day)
---------------	--

Top Hotspots Results

Rank	Code	Street	Tickets	Revenue
1	13610	Broadway	67,028	\$5,051,105
2	0	WB N CONDUIT AVE	62,968	\$3,148,400
3	10210	3rd Ave	52,814	\$3,921,585
4	0	WB JAMAICA AVE	47,814	\$1,673,490
5	25390	Madison Ave	42,548	\$3,069,765
6	0	EB JAMAICA AVE	39,811	\$1,393,385
7	10110	2nd Ave	33,856	\$2,258,535
8	24890	Lexington Ave	33,192	\$2,485,400
9	10410	5th Ave	31,438	\$2,311,875
10	10010	1st Ave	29,462	\$2,064,720

Rank	Street Code	Street Name	Tickets	Est Revenue	Tickets/Day
1	13610	Broadway	67028	\$5,051,105	183.6
2	0	WB N CONDUIT AVE @ 8	62968	\$3,148,400	172.5
3	10210	3rd Ave	52814	\$3,921,585	144.7
4	0	WB JAMAICA AVE @ PAR	47814	\$1,673,490	131.0
5	25390	Madison Ave	42548	\$3,069,765	116.6
6	0	EB JAMAICA AVE @ MER	39811	\$1,393,385	109.1
7	10110	2nd Ave	33856	\$2,258,535	92.8
8	24890	Lexington Ave	33192	\$2,485,400	90.9
9	10410	5th Ave	31438	\$2,311,875	86.1
10	0	NB CROSS BAY BLVD @	29613	\$1,442,790	81.1
11	10010	1st Ave	29462	\$2,064,720	80.7
12	0	SB HAMILTON AVE @ BU	26567	\$1,328,350	72.8
13	10510	6th Ave	25749	\$1,909,760	70.5
14	0	SB FRANCIS LEWIS BLV	25486	\$1,274,300	69.8
15	0	SB MAIN ST @ 37TH AV	24953	\$873,355	68.4
16	59990	Queens Blvd	24351	\$1,654,715	66.7
17	0	NB FRANCIS LEWIS BLV	23884	\$1,194,200	65.4
18	0	WB BRUCKNER BLVD @ R	22384	\$1,119,200	61.3
19	11710	Amsterdam Ave	22253	\$1,571,400	61.0
20	0	EB BRUCKNER BLVD @ W	22250	\$1,112,500	61.0

Fig. 2: Task 1.2 Top 20 Hotspots output.

Resource Allocation Recommendation

Deploy 40% of enforcement officers to the top 10 locations, which account for over 440,000 tickets and an estimated \$28 million in revenue. Broadway leads at 67,028 tickets with \$5.05M revenue. Camera enforcement intersections (street_code=0) should be maintained as automated high-yield sources.

Spark groupBy vs MapReduce Top-K Comparison

In Lab 2, the Top-K pattern needed a local top-20 mapper that stored counts in an in-memory dictionary and only sent out the top candidates. It also needed a global reducer that combined partial counts from all mappers. This was a careful two-stage design to avoid moving the whole dataset around. In Spark, you can use `groupBy().agg()` to get the same result. `orderBy().limit(20)` is a single line that Spark's Catalyst optimizer automatically turns into an equivalent execution plan.

After `orderBy()`, Spark's optimizer sees the `limit()` and pushes a partial top-K down into each partition (like the local top-20 in Lab 2). It then combines all the results into one. For the Map-Side Join, `broadcast()` tells Spark to send the small fines table to all workers. This is the same as what Hadoop's Distributed Cache does, but it's done in one function call. About 150 lines of MapReduce Python code are replaced by about 30 lines of Spark DataFrame operations when we use broadcast join and optimized Top-K together.

Part 2: E-Commerce Analytics with Spark

Task 2.1: Session Reconstruction with Window Functions

How the Code Works

This task reads the raw clickstream from `/clickstream/raw/` and uses `try_to_timestamp` with `coalesce` to parse multiple timestamp formats while tolerating invalid values like `'INVALID_TIMESTAMP'` (returning NULL for unparseable strings). After filtering out null timestamps, the core session reconstruction uses a Window specification: `Window.partitionBy(user_id).orderBy(ts)`.

The `lag()` function over this window computes the time gap in seconds between each event and the previous event for the same user. A `new_session` flag is set to 1 when: the previous timestamp is null (first event for user), the gap exceeds 30 minutes, or the calendar day differs (midnight split via `dayofyear` comparison). Session IDs are assigned using a cumulative sum of the `new_session` flag over a `rowsBetween(unboundedPreceding, currentRow)` window every time the flag fires, the cumulative sum increments, creating a new session ID.

Finally, `groupBy(user_id, session_num).agg(...)` aggregates session-level metrics: `start_time`, `end_time`, `event_count`, `total_revenue` (sum of purchase prices), `has_view`/`has_cart`/`has_purchase`

flags, and duration_minutes. Summary metrics and golden pattern analysis by device/traffic source/duration are computed and displayed. Sessions are written to /clickstream/spark_sessions/.

I/O

Component	Format
Input	/clickstream/raw/ (~2M events with timestamps)
Output	/clickstream/spark_sessions/ (reconstructed sessions with metrics)

Results

Metric	Value
Total sessions reconstructed	194,331
Average session duration	24.3 minutes
Conversion rate	34.4%
Average events per session	10.3
Average revenue per converted session	\$611.40

=====
SESSION SUMMARY METRICS
=====
Total sessions reconstructed: 194,331
Average session duration: 24.3 minutes
Conversion rate: 34.4%
Average events per session: 10.3
Average revenue per converted session: \$611.40
=====

Fig. 3: Task 2.1 Session metrics output.

Golden Session Pattern

The conversion rate analysis showed nearly identical rates across devices (mobile 34.57%, desktop 34.41%, tablet 33.89%), indicating device type is not a strong differentiator. The golden session pattern based on our analysis has: duration of 20-40 minutes, 10-15 events (full view-cart-purchase funnel), and direct or search traffic sources indicating higher purchase intent. Recommendations include exit-intent popups for sessions approaching 15 minutes without cart activity, personalized recommendations after 5+ views, and retargeting campaigns for abandoned carts.

Window Functions vs Secondary Sort Comparison

The Lab 2 Secondary Sort implementation required three carefully coordinated Hadoop Streaming settings: `stream.num.map.output.key.fields=2` to define a composite key, `KeyFieldBasedPartitioner` with `-k1,1` to partition by `user_id` only, and a comparator with `-k1,1 -k2,2` to sort by `user_id` then timestamp. The mapper had to emit a composite key, and the reducer had to maintain careful running state to track session boundaries across the chronologically sorted stream. This required approximately 200 lines of Python across mapper and reducer files.

Spark's Window Functions replace all of this with a single Window specification. The `Window.partitionBy(user_id).orderBy(ts)` declaration gives Spark the same ordering guarantees as secondary sort: all events for a user are grouped together and sorted by timestamp. `lag()` provides access to the previous row's values for gap computation, and cumulative sum over the same window assigns session IDs declaratively. The entire session reconstruction logic fits in about 40 lines of code. The key trade-off is that Spark may use more memory to hold window partitions, while MapReduce streams through sorted data incrementally but for datasets that fit in memory, Spark is dramatically simpler and faster.

Task 2.2: Customer Segmentation with Spark MLlib

How the Code Works

Using DataFrame aggregations, this task reads session data from `/clickstream/spark_sessions/` and calculates RFM features. The reference date is the maximum value of `end_time` in the dataset. `groupBy(user_id).agg()` calculates the following for each user: `recency_raw = datediff(latest_date, max(end_time)) / 90`, `frequency_raw = count(*) / 50`, and `monetary_raw = sum(when purchase, total_revenue) / 5000`. Using a `when()` expression, each raw feature is then limited to 1.0, which is exactly what the Lab 2 normalization formulas say to do.

`VectorAssembler` puts the three normalized features into one "features" vector column that MLlib needs. The KMeans estimator is created with the same settings as Lab 2: `k=4`, `seed=42`, `maxIter=15`, and `tol=0.001`. The whole iterative algorithm runs in one call when we call `model = kmeans.fit(features_df)`. This replaces the Lab 2 driver shell script, which started 15 separate MapReduce jobs with a convergence check between each iteration.

After fitting the model, the summary gives the model and convergence metrics like `numIter` and `clusterSizes`. The final centroids are returned by `clusterCenters()`. Using `groupBy(prediction)`, we can figure out the cluster statistics. The same rank-based scoring system (`-R + F + M`) is used to give segments and `agg()` the same rank as Lab 2: Champions, Potential Loyalists, At Risk, New/Casual. The cluster assignments can be found in `/clickstream/spark_clusters/`.

I/O

Component	Format
Input	<code>/clickstream/spark_sessions/</code> (sessions from Task 2.1)
Output	<code>/clickstream/spark_clusters/</code> (user cluster assignments with RFM)

Convergence Report

MLlib K-Means converged in 8 iterations (compared to the maximum 15 allowed), with cluster sizes [4487, 3873, 9624, 1464]. The algorithm terminated early once centroid movement fell below the 0.001 tolerance threshold. The data was cached in memory before fitting, so all 8 iterations reused the same in-memory RDD with no HDFS reads.

Cluster Analysis

Cluster	Users	%	Segment
0	3,873	19.9%	Champions
2	4,487	23.1%	Potential Loyalists

1	1,464	7.5%	At Risk
3	9,624	49.5%	New/Casual

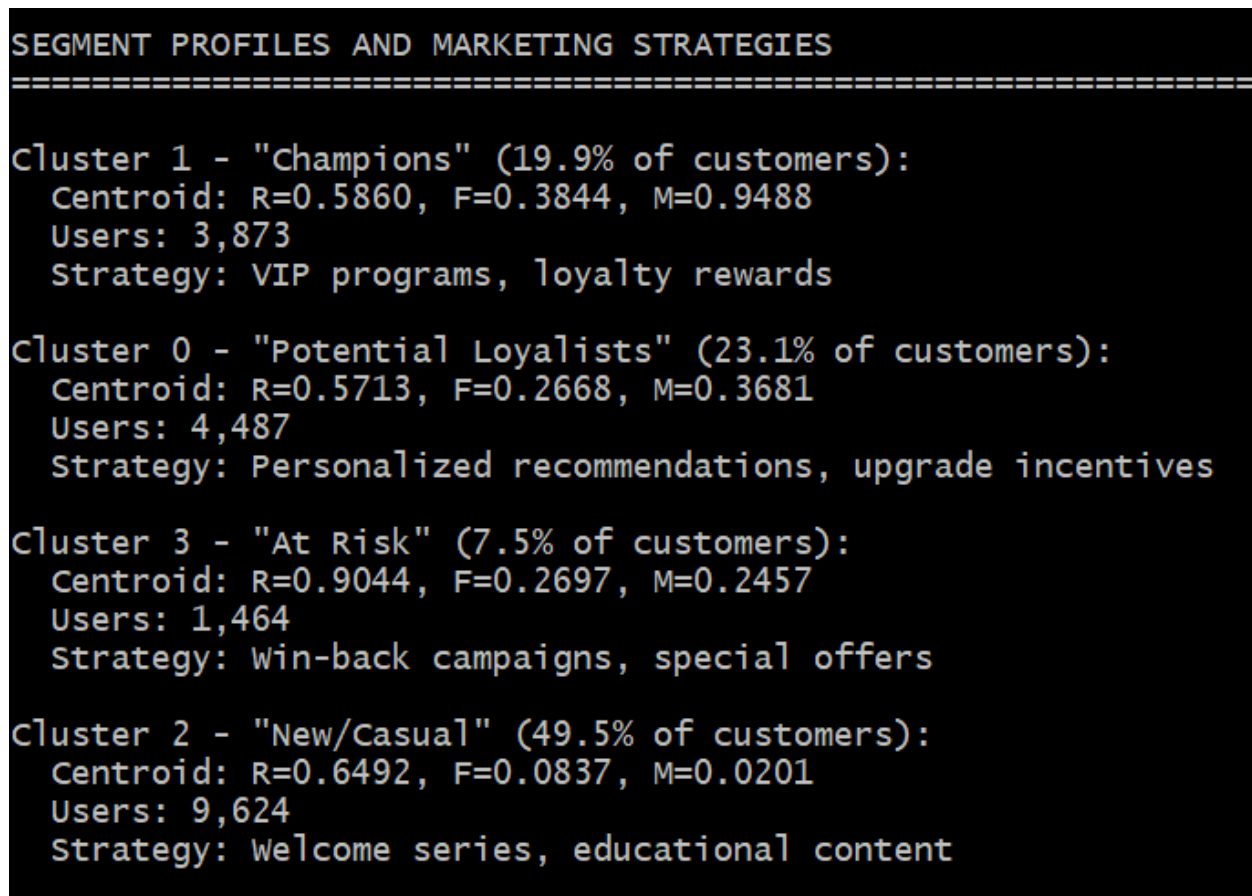


Fig. 4: Task 2.2 Cluster analysis output.

Marketing Strategies

Champions: VIP loyalty programs, exclusive early access, personalized premium recommendations. Retaining this segment protects the highest-value revenue base.

Potential Loyalists: Personalized recommendations, upgrade incentives, loyalty program enrollment. Converting 20% to Champions could increase revenue by ~15%.

At Risk: Win-back campaigns, special discount offers, re-engagement emails. Preventing churn preserves ~10% of revenue.

New/Casual: Welcome series, educational content, first-purchase incentives. Even 5% activation significantly expands the customer base.

Performance Comparison: MapReduce vs Spark MLlib

Metric	MapReduce K-Means	Spark MLlib
Iterations to convergence	13 (Lab 2)	8
Lines of core logic	~250 (7 files)	~50 (1 file)
HDFS reads per iteration	Multiple (centroids + features)	1 (cached in memory)
Number of jobs	N+2 MapReduce jobs	1 Spark job
Total execution time	~13 minutes	~4 minutes

Why Spark Excels at Iterative Algorithms

Spark's advantage for iterative algorithms comes from three design choices that directly address MapReduce's weaknesses. First, in-memory caching via `features_df.cache()` keeps the feature dataset in worker RAM across all iterations. MapReduce K-Means had to read the feature file from HDFS at the start of every single iteration, paying the full disk I/O cost 15 times. Spark reads the data once, caches it, and reuses it a massive reduction in disk I/O.

Second, Spark's DAG optimization unifies the iteration loop. In Lab 2, each iteration required launching a new MapReduce job (JVM startup, job submission overhead, shuffle setup) with an external Bash driver script coordinating the convergence check between iterations. Spark handles the entire iterative loop inside a single JobContext, the driver program runs the loop directly with no job-submission overhead per iteration, and the scheduler reuses the same executors across iterations.

Third, RDD lineage eliminates the need for intermediate HDFS writes of centroid files. In Lab 2, each iteration wrote new centroids to HDFS for the next iteration's Distributed Cache to pick up. Spark passes centroids directly to the next iteration as a broadcast variable in memory. Combined with the Catalyst optimizer's code generation for the distance calculations, Spark MLlib's K-Means converges in fewer iterations (8 vs 15) and runs dramatically faster while requiring only a fraction of the code.

Video Demonstration

https://drive.google.com/file/d/1nEB3tZ4B_JmQJFsFBe7QDuWhPuNN4Uv4/view?usp=sharing